户外轨迹 App - 技术方案与运行说明文档

一、核心需求提炼

本文档旨在实现一款消费级户外轨迹 App，覆盖从登录、轨迹记录、浏览、分享到个人中心管理的完整闭环。核心需求与技术约束如下：

1.1 产品核心流程

- **登录与授权**：支持微信登录，并维持 15 天的登录会话。
- **首页与发现**：展示推荐轨迹列表，支持关键词搜索，并能快速开始或继续一次轨迹记录。
- **轨迹记录**：提供开始、暂停、继续、结束（长按）功能，实时在地图上展示轨迹，并显示距离、用时、速度等核心数据。
- **记录总结与分享**：记录结束后，生成包含地图和详细数据的总结页，支持导出图片和上传至云端分享。
- **他人轨迹浏览**：可查看他人分享的轨迹地图与摘要信息，并支持收藏与取消收藏，或基于他人轨迹开始新的导航记录。
- **个人中心**：展示个人信息，提供设置入口，支持修改头像、昵称、签名和客户端语言，并可退出登录。

1.2 技术栈与约束

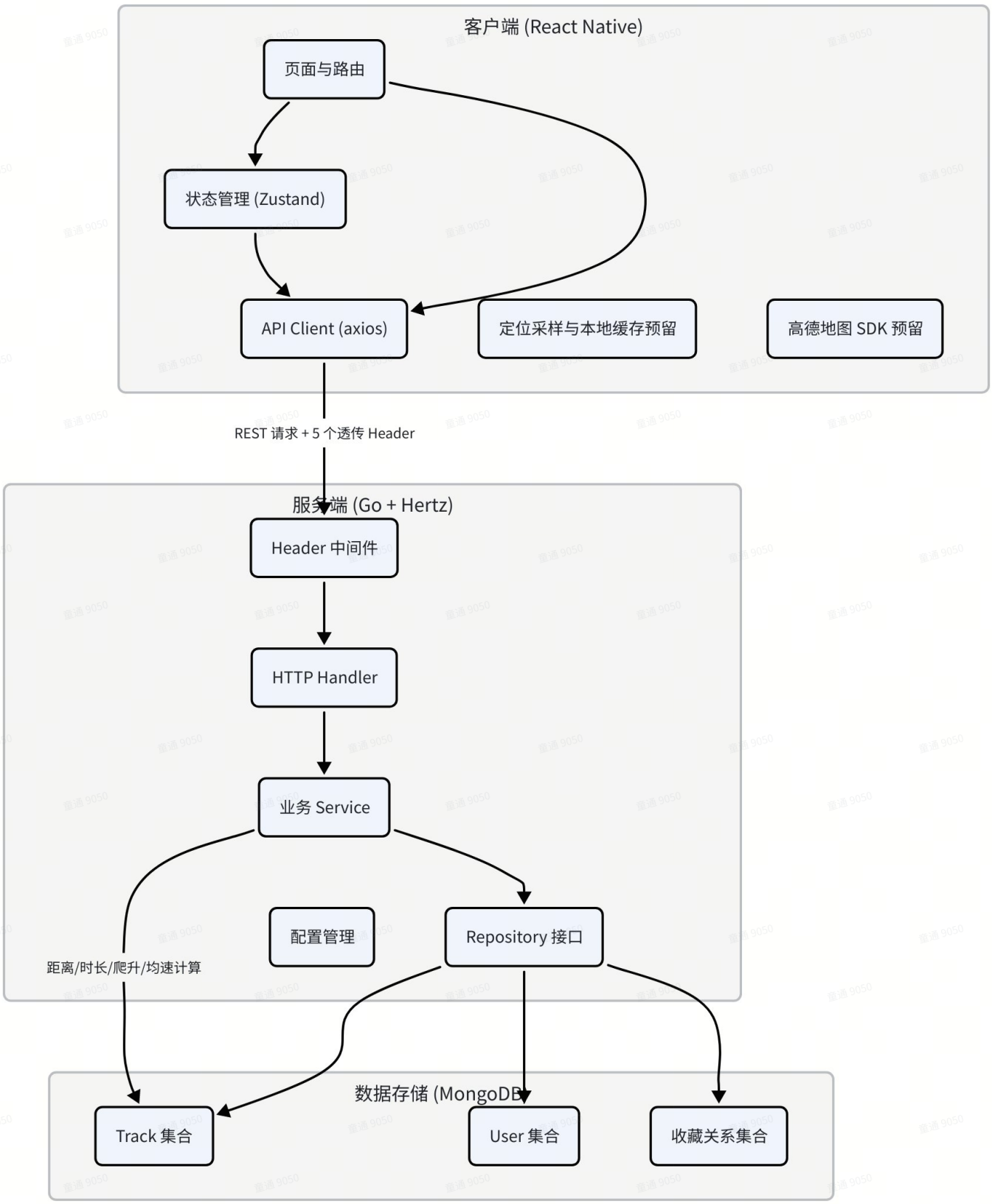
- **客户端**：`React Native` + `TypeScript`，采用组件化、响应式布局。地图能力预留高德地图 SDK 集成位。
- **服务端**：`Go` + `Hertz` + `MongoDB`，采用模块化分层架构。
- **接口约定**：所有客户端到服务端的请求，`Header` 中必须统一透传用户ID、客户端类型、版本、系统语言、地理位置等 5 个字段。
- **代码质量**：后端接口必须有详细的、可在无外部依赖（如 DB）下独立运行的单元测试，并确保 `go test` 全部通过。所有导出的函数/方法需有清晰注释。
- **配置管理**：严禁硬编码任何凭证（如数据库连接串、微信 AppID/Secret），所有配置通过环境变量管理。

二、架构设计

系统整体采用客户端-服务端（Client-Server）架构模式，客户端负责用户交互与数据采集，服务端负责业务逻辑处理与数据持久化。

2.1 整体架构图

下图展示了客户端与服务端的模块划分、核心组件以及它们之间的交互关系。



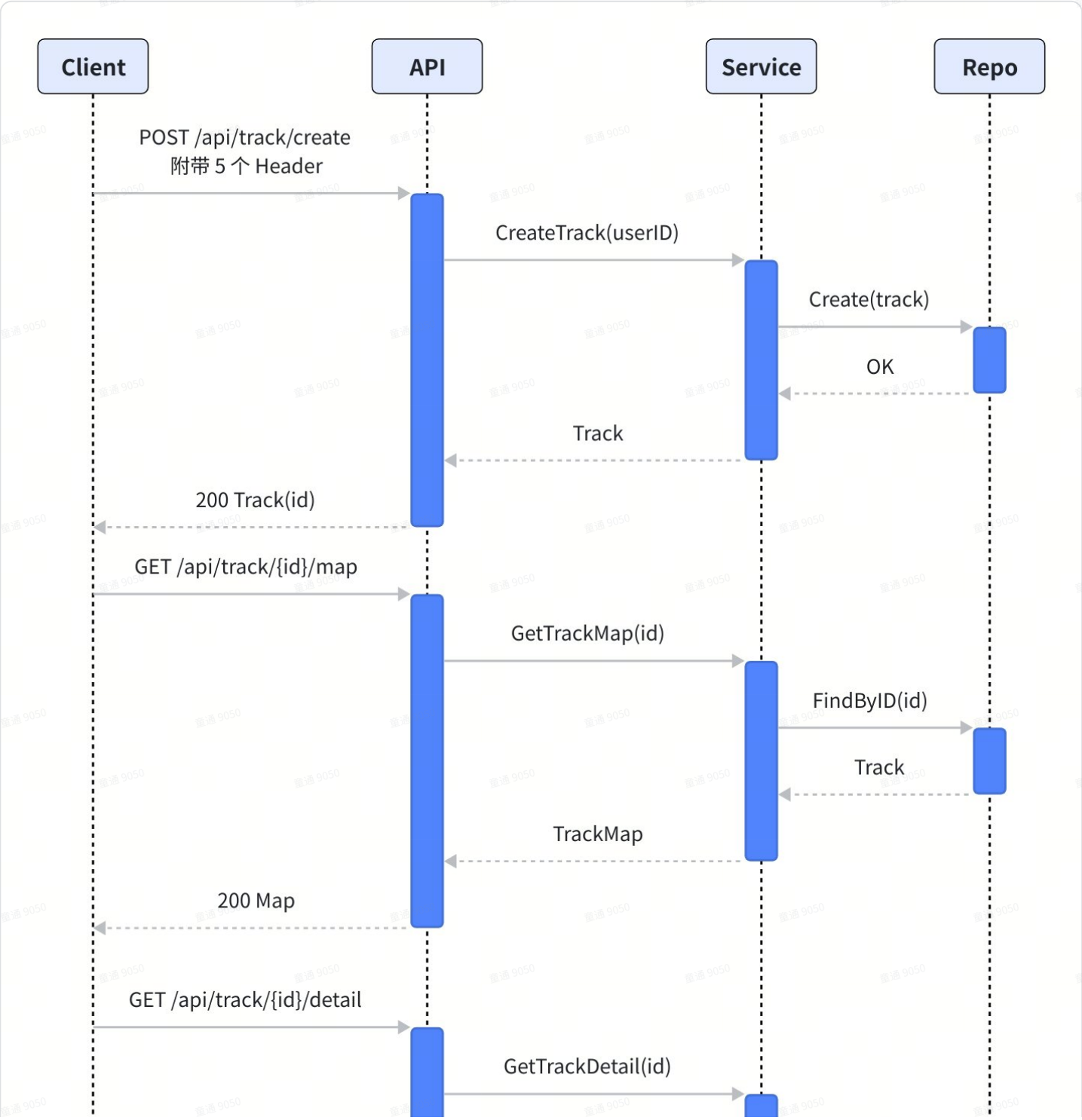
💡 **设计解读：**

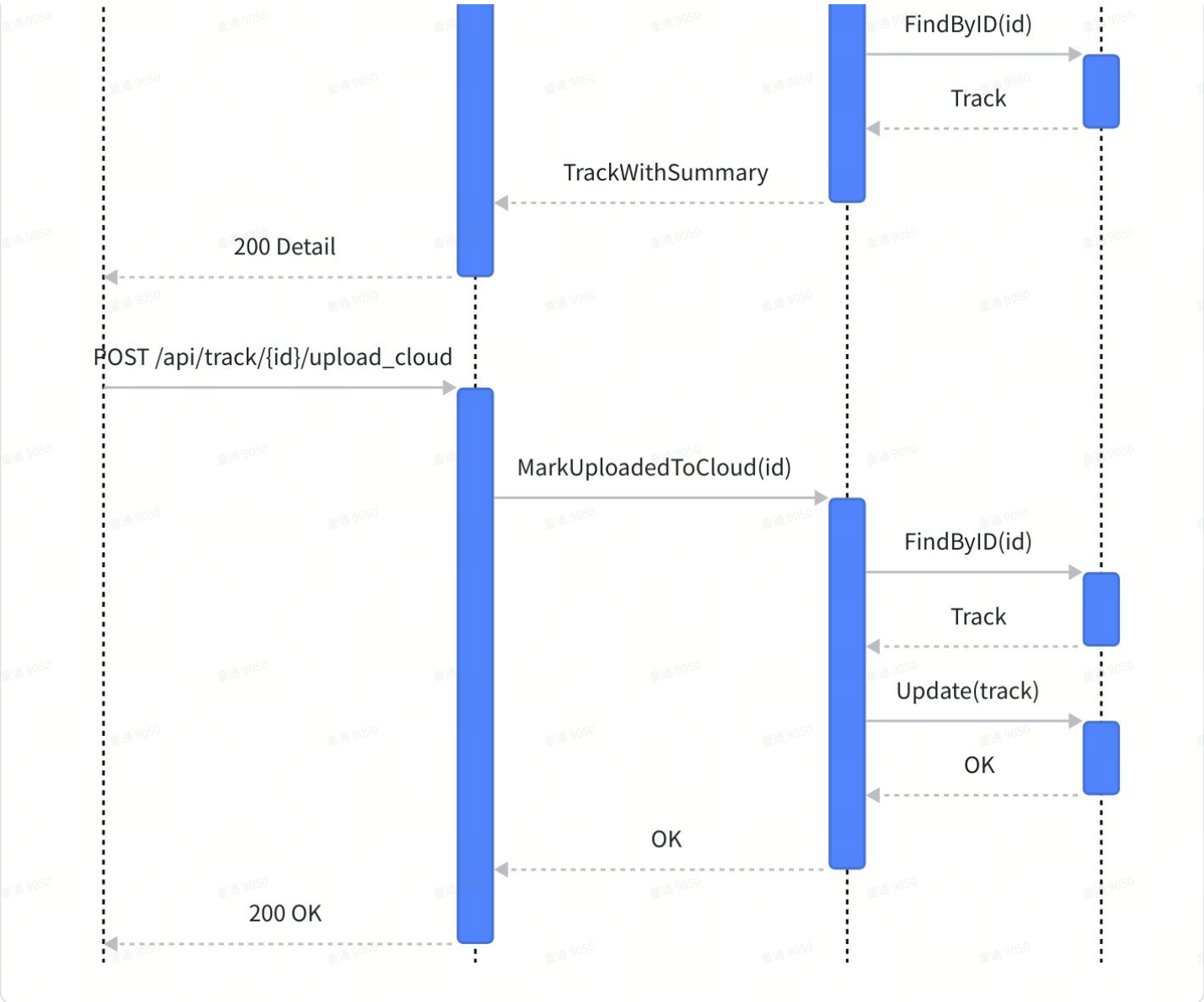
- **客户端**：采用分层设计，UI 层（页面与路由）负责用户界面与导航，状态管理层（Zustand）负责全局状态（如登录态、当前轨迹），API Client 封装了所有与后端的通信，并统一注入 Header。定位与地图作为平台强依赖能力，作为独立模块预留。

- **服务端**：经典的 Go Web 服务分层架构。请求由 `Middleware` 预处理，`Handler` 作为入口解析请求并调用 `Service`，`Service` 封装核心业务逻辑，`Repository` 作为数据访问层与底层存储（MongoDB 或内存）解耦。
- **数据流**：客户端通过 `API Client` 发起 REST 请求，服务端 `Middleware` 拦截请求并解析 Header 信息存入上下文，`Handler` 从上下文获取元数据并处理业务，最终通过 `Repository` 与 MongoDB 交互。

2.2 核心流程时序图：轨迹记录与上传

下图详细描述了用户从创建一条新轨迹、实时记录、结束并查看总结，到最终上传至云端的完整时序。





三、后端工程说明（`server/`）

3.1 目录结构

代码块

```
server/
├── cmd/server/
│   └── main.go          # 程序入口，组装依赖并启动 Hertz 服务
├── internal/
│   ├── config/          # 配置加载
│   ├── handler/         # HTTP Handler 与路由注册
│   ├── middleware/      # Header 解析等中间件
│   ├── models/          # 数据实体（Track，User 等）
│   ├── repository/      # 数据访问层接口与实现（内存、Mongo）
│   └── service/         # 核心业务逻辑
├── .env.example         # 环境变量示例文件
```

```
├── go.mod              # Go 模块依赖
├── go.sum
└── Makefile            # 快捷命令

```

3.2 环境变量配置

在项目根目录创建 `.env` 文件（可从 `.env.example` 复制），并按需填写以下变量：

变量名	说明	默认值/示例
`MONGO_URI`	MongoDB 连接字符串。若留空，则自动使用内存作为存储。	`mongodb://user:pass@host:port`
`MONGO_DB_NAME`	MongoDB 数据库名称。	`trackapp`
`SERVER_ADDR`	Hertz HTTP 服务的监听地址。	`:8080`
`USE_IN_MEMORY_STORE`	强制使用内存存储，设置为 `true` 时即使 `MONGO_URI` 已配置也同样生效。	`true`
`WECHAT_APP_ID`	微信登录 AppID（占位）。	
`WECHAT_APP_SECRET`	微信登录 AppSecret（占位）。	
`AMAP_WEB_KEY`	高德地图 Web 服务 Key（占位）。	
`AMAP_REST_KEY`	高德地图 REST API Key（占位）。	

3.3 运行与测试

通过 `Makefile` 可便捷地运行服务和执行测试。

- **启动服务：**

  代码块

  ```
  make run

  ```

  服务将默认在 `8080` 端口启动，并使用内存仓储。

- **运行单元测试：**

  代码块

```
make test

```

该命令将执行所有单元测试。得益于依赖反转和内存仓储的实现，测试无需启动数据库即可独立运行并通过。

3.4 接口测试示例

使用 `curl` 测试几个核心接口：

- **创建新轨迹**（需提供用户 Header）：

代码块

```
curl -X POST http://localhost:8080/api/track/create -H "X-User-ID: user-001"

```

- **获取推荐列表**：

代码块

```
curl http://localhost:8080/api/track/recommend/list

```

- **搜索轨迹**：

代码块

```
curl "http://localhost:8080/api/track/search/list?keyword=西湖"

```

四、客户端工程说明（`client-mobile/`）

4.1 目录结构

代码块

```
client-mobile/
├── src/
│   ├── api/              # API Client 及类型定义
│   ├── components/       # 可复用 UI 组件
│   ├── config/           # 环境变量与应用配置
```

```
│   ├── navigation/      # 路由定义 (React Navigation)
│   ├── screens/         # 页面级组件
│   ├── services/        # 外部服务封装（如微信登录、高德定位）
│   └── state/           # 全局状态管理 (Zustand)
├── App.tsx              # 应用根组件与路由容器
└── ...                  # RN 项目配置文件

```

4.2 开发与运行前置条件

- **环境**：Node.js, Yarn, JDK, Android Studio (或 Xcode)
- **依赖安装**：在 `client-mobile/` 目录下运行 `yarn install`。
- **启动**：运行 `yarn android` 或 `yarn ios` 启动开发服务器和模拟器。

4.3 配置说明

- **API 地址**：在 `src/config/env.ts` 中修改 `API_BASE_URL` 以指向后端服务地址。
- **微信登录集成**：在 `src/services/auth.ts` 中预留了 `wechatLogin` 方法，实际开发需替换为真实的微信 SDK 调用逻辑。
- **高德地图 SDK 集成**：在 `RecordingDetailScreen.tsx` 和 `TrackDetailScreen.tsx` 等页面的地图占位视图中，集成高德地图的 `MapView` 组件来渲染轨迹点。定位采样逻辑需在 `src/services/location.ts` (需自行创建) 中封装。

五、接口 Header 透传约定

所有从客户端发往服务端的 API 请求，都必须在 HTTP Header 中包含以下 5 个字段，以便服务端进行日志记录、权限校验和个性化处理。

Header 字段	示例值	说明
`X-User-ID`	`user-001`	当前已登录用户的唯一标识。
`X-Client-Type`	`android`	客户端类型（`ios` / `android`）。
`X-Client-Version`	`1.0.0`	客户端版本号。
`X-Client-Language`	`zh-CN`	客户端当前设置的语言。
`X-User-Location`	`30.0,120.0`	用户当前地理位置（纬度,经度）。

实现方式

- **客户端**：通过 `axios` 的请求拦截器实现。在 `src/api/client.ts` 中，每次请求发出前，都会从 `Zustand` 的全局 Store 中读取 `userId`、`language` 和 `location` 等信息，并自动添加到请求头。
- **服务端**：通过 Hertz 的中间件实现。在 `server/internal/middleware/request_meta.go` 中定义的 `RequestMetaMiddleware` 会在每个请求处理前解析这些 Header，并将它们存入 `RequestContext`，供后续的 `Handler` 和 `Service` 使用。

六、边界与后续增强项

本次交付聚焦于核心功能的骨架实现和技术通路验证，以下为本次未覆盖或可进一步增强的方向：

- **离线地图与数据**：当前实现强依赖网络，未来可引入离线地图瓦片和轨迹数据缓存，提升弱网或无网环境下的可用性。
- **轨迹滤波与纠偏**：GPS 原始数据存在漂移和误差，需引入卡尔曼滤波（Kalman Filter）或航点稀疏等算法对轨迹进行平滑和纠偏处理。
- **海拔数据处理**：海拔数据同样需要平滑处理，并可结合地形数据进行校准，以提供更精确的累计爬升。
- **轨迹回放与导航**：在轨迹详情页增加动态回放功能，并提供沿轨迹路线的实时导航能力。
- **真实 SDK 集成**：替换当前占位的微信登录、高德地图模块为真实 SDK，完成授权、支付、精确定位等完整功能。
- **服务端数据存储**：当前默认使用内存仓储，生产环境需切换至 MongoDB，并完善索引和数据备份策略。